DSC613 Big Data Analytics Assignment

Large-Scale Data Analysis Using Apache Hive and Visualization

This 100-mark postgraduate assignment asks you to analyse a 15,000-row e-commerce transaction dataset with Apache Hive, then visualise and interpret the results. You write and run every HiveQL query yourself; this brief gives you the dataset, the tasks, what each output should look like, and a few starter snippets.

1. Overview and Scenario

Theme: Large-Scale Data Analysis Using Apache Hive and Visualization.

An e-commerce company generates millions of transaction records every month. Management wants a Big Data platform to analyse customer behaviour, product performance, sales trends, payment preferences and order fulfilment. You are the data analyst. You will process the transaction data in Apache Hive and produce analytical results that can be visualised for management.

The 15,000-row file you receive is a small sample of that data. Write every query as if it will later run on billions of rows.

2. Learning Objectives

By completing this assignment you will be able to:

1. Explain the role of Apache Hive in Big Data processing.
2. Create a database and table in Hive.
3. Load a large structured dataset into Hive.
4. Clean and transform data using HiveQL.
5. Use GROUP BY, JOIN, ORDER BY, HAVING, aggregate functions and conditional expressions.
6. Derive meaningful analytical insights from the dataset.
7. Export Hive query results for visualisation.
8. Create appropriate visualisations and interpret them.
9. Explain how distributed processing through Hive helps large-scale analysis.

3. Assignment Structure

Part	Focus
A	Hive environment setup: database, table, data load, validation
B	Data exploration and cleaning
C	HiveQL analytics (20 questions)
D	Visualisation
Report	Interpretation, Big Data perspective, organization

4. Conditions

- Pandas, Matplotlib, Seaborn, Power BI or Tableau may be used only in Part E, on files exported from Hive.

- Every query in your report must be copied from your `.hql` script and show its real output.

5. Dataset and Data Dictionary

The dataset is the file `ecommerce_transactions.csv`: 15,000 synthetic transactions placed between 1 January 2024 and 31 December 2025 (24 months), with one header row and one transaction per line. Amounts are in Indian rupees (INR). Dates use the format `yyyy-MM-dd`. The file is comma-separated with Unix (LF) line endings.

The data is realistic but not clean. Missing values, out-of-range values and inconsistent text formatting have been placed in it on purpose. Finding them is part of Part B, and this brief does not list them.

5.1 Data dictionary

Field	Description	Values and format
transaction_id	Unique transaction identifier	`TXN` followed by a number, e.g. `TXN100001`
transaction_date	Date the order was placed	`yyyy-MM-dd`, 2024-01-01 to 2025-12-31
customer_id	Customer identifier	`CUST` followed by 5 digits; a customer can place many orders
customer_age	Customer age in years	Whole number; a valid age is 18 to 100
gender	Customer gender	Male, Female, Other
city	Customer city	14 Indian cities, e.g. Mumbai, Delhi, Kochi
product_category	Category of the product bought	8 categories, e.g. Electronics, Fashion, Grocery
product	Product name	44 distinct products; each belongs to one category
quantity	Units purchased in the order	Whole number, 1 to 5
unit_price	Price per unit in INR	Decimal with 2 places; the same product can have different prices (discounts)
payment_method	How the customer paid	Credit Card, Debit Card, UPI, Cash, Net Banking
order_status	Final state of the order	Completed, Cancelled, Returned
customer_rating	Rating given by the customer	Whole number, 1 to 5
delivery_days	Days from order to delivery	Whole number, at least 1 for a delivered order

5.2 Business definitions (use these in every query)

Term	Definition
transaction_value	`quantity * unit_price` for one row
Revenue	Sum of transaction_value for rows with order_status = `Completed` only. Cancelled and Returned orders earn no revenue.
Gross order value	Sum of transaction_value over all statuses. Use it only when a question says so.
Order	One row of the file (one transaction)
Return rate	Returned orders divided by all orders, as a percentage
Cancellation rate	Cancelled orders divided by all orders, as a percentage

A blank `customer_rating` or `delivery_days` is legitimate for a Cancelled order, because nothing was delivered. It is a data problem for a Completed or Returned order. State your treatment of each case in your report.

5.3 Dataset file

The CSV is supplied with this brief. Do not edit it by hand in Excel, because Excel can change date formats and strip leading characters. Load the original file into Hive as supplied.

6. Part A: Hive Environment Setup

By the end of Part A you have a Hive database, a raw table holding all 15,000 rows with correct data types, and evidence that the load worked.

6.1 Setting up Hive

Use any one of the three options below and state which one you used in your README.

Option 1: Docker (recommended for a laptop). The Apache Hive project publishes an official image that runs HiveServer2 with an embedded metastore.

```
docker run -d -p 10000:10000 -p 10002:10002 --env SERVICE_NAME=hiveserver2 --name hive4 apache/hive:4.0.
0
docker cp ecommerce_transactions.csv hive4:/tmp/ecommerce_transactions.csv
docker exec -it hive4 beeline -u 'jdbc:hive2://localhost:10000/'
```

This single container stores data on its local file system, not in HDFS. Mention that in your Big Data discussion.

Option 2: Hadoop cluster or university lab with Hive and HDFS. Put the file into HDFS first, then load it from there.

```
hdfs dfs -mkdir -p /user/<your_username>/ecommerce/raw
hdfs dfs -put ecommerce_transactions.csv /user/<your_username>/ecommerce/raw/
hdfs dfs -ls /user/<your_username>/ecommerce/raw/
beeline -u jdbc:hive2://<host>:10000/
```

Option 3: A vendor sandbox or VM (for example a Cloudera or Hortonworks quickstart VM) that already has Hive. Follow its documentation and note the Hive version with `SELECT version();` or `hive --version`.

6.2 Starter HiveQL

Complete every blank. Choose each data type yourself and justify the choices in your report.

```
CREATE DATABASE IF NOT EXISTS ____________;
USE ecommerce_analytics;

CREATE TABLE IF NOT EXISTS transactions_raw (
    transaction_id    STRING,
    transaction_date  ______,
    customer_id       STRING,
    customer_age      ______,
    gender            STRING,
    city              STRING,
    product_category  STRING,
    product           STRING,
    quantity          ______,
    unit_price        ______,
    payment_method    STRING,
    order_status      STRING,
    customer_rating   ______,
    delivery_days     ______
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '___'
STORED AS ________
TBLPROPERTIES ('skip.header.line.count'='1');

LOAD DATA LOCAL INPATH '/path/to/ecommerce_transactions.csv'
INTO TABLE transactions_raw;
```

6.3 Tasks

#	Task	Expected output
A1	Create the database `ecommerce_analytics` and switch to it.	`SHOW DATABASES;` lists `ecommerce_analytics`.
A2	Create the raw table with a suitable data type for every one of the 14 attributes.	`DESCRIBE transactions_raw;` shows 14 columns with the types you chose.
A3	Load the CSV into the table. Explain in one sentence what `LOCAL` changes in `LOAD DATA`.	The statement completes without error.
A4	Show the first 10 rows.	10 rows whose values sit in the right columns, with no column entirely NULL.
A5	Count the rows.	One number equal to the number of data lines in the CSV, not counting the header.
A6	Run `DESCRIBE FORMATTED transactions_raw;` and report the table type, location and input format.	A screenshot or text output, plus 2 to 3 sentences on what the location tells you about where Hive stores the data.
A7	Check that the types work: pick three columns of different types and show that a numeric comparison, a date function and a string function each return sensible results.	Three short queries with outputs.

6.4 Marking guide for Part A

- **Database and table creation:** correct database name, 14 columns, sensible types, correct delimiter and header handling, and a brief justification of each type choice.
- **Data loading and validation:** successful load, a row-count check against the source file, a sample of rows, and evidence you inspected for load problems (for example a column that is all NULL).

6.5 Things to watch

- If the count is one higher than the file's data rows, the header was not skipped.
- A numeric or date column that reads back as NULL when the CSV has a value usually means the type or delimiter is wrong.
- For numeric columns, Hive reads a blank field as NULL. For STRING columns, a blank field stays an empty string, which is not NULL. Remember this in Part B.

7. Part B: Data Exploration and Cleaning

Part B ends with a cleaned table or view, `transactions_clean`, that every later query uses. Do not analyse the raw table in Parts C and D.

7.1 Starter example

This pattern counts NULLs in one column. Extend the idea yourself to every column that needs it.

```
SELECT
    COUNT(*)                                                AS total_rows,
    SUM(CASE WHEN customer_age IS NULL THEN 1 ELSE 0 END) AS null_age
FROM transactions_raw;
```

NULL is not the only way a value can be missing or wrong. Look for empty strings, values outside the allowed range in the data dictionary, and the same real-world value written in different ways.

7.2 Tasks

#	Task	Expected output
B1	Count the total records.	One number. It must match your Part A count.
B2	Count missing values in every column. Treat NULL and empty or whitespace-only strings as missing.	One row, or a short table, with a missing-value count per column.
B3	Find invalid values: ages outside 18 to 100, ratings outside 1 to 5, quantity below 1, non-positive prices, and delivery_days below 1 on delivered orders.	A count per rule, plus a few sample offending rows for each rule that returns rows.
B4	List the distinct values of gender, city, payment_method and order_status with their counts.	Lists that expose spelling, case or spacing inconsistencies. A column with inconsistent values shows more distinct values than the data dictionary allows.
B5	Count unique customers.	One number. Explain in one sentence why it is far smaller than the row count.
B6	Calculate the minimum, maximum and average of transaction_value, using the formula `quantity * unit_price`.	One row with three numbers. State whether you used all statuses or only Completed, and why.
B7	Decide how to handle each kind of problem you found: drop the row, keep the row and set the value to NULL, replace it with a label such as `Unknown`, or impute it.	A table with columns *Column, Problem, Rows affected, Treatment, Reason*. Every treatment needs a reason.
B8	Create the cleaned view or table `transactions_clean`, with all standardisation rules applied and the new column `transaction_value`.	`DESCRIBE transactions_clean;` shows the 14 original columns (types possibly adjusted) plus `transaction_value`.
B9	Validate the clean data: re-run the B2 to B4 checks on `transactions_clean`.	Evidence that invalid values and inconsistent text are gone, plus a raw-versus-clean row count with an explanation of any difference.
B10	Explain when a table is a better choice than a view for cleaned data, and say which you chose.	3 to 4 sentences that mention recomputation cost on large data.

7.3 Guidance on missing values

- Do not delete rows just because a rating or delivery time is missing. A Cancelled order never has either, and deleting those rows would corrupt the cancellation and return-rate questions in Part C.
- Do not replace a missing rating with 0, because that pulls every average down.
- When you standardise text, apply the same rule to every column that needs it, and show the distinct values before and after.

7.4 Marking guide for Part B

- Completeness of the exploration (B1 to B6).

- Whether each cleaning rule is justified and appropriate to the field (B7).
- A working `transactions_clean` containing `transaction_value`, with evidence that the cleaning worked (B8 to B10).

8. Part C: HiveQL Analytics

Answer all 20 questions with queries on `transactions_clean`. Unless a question says otherwise, revenue means Completed orders only (section 5.2). For each question, put the query in your `.hql` script, save the output, and write one sentence stating the answer.

The dataset covers two years, so group by year and month together. Grouping by month number alone merges January 2024 with January 2025.

8.1 Questions

#	Question	Required HiveQL features	Expected output
C1	What is the total revenue?	`SUM`, `WHERE`	One number in INR.
C2	What is the total number of transactions, and how are they split by order status?	`COUNT`, `GROUP BY`	A total, plus one row per status (3 rows) with a count.
C3	What are the top 10 product categories by revenue?	`SUM`, `GROUP BY`, `ORDER BY`, `LIMIT`	A table of categories ranked by revenue, with each category's share of total revenue. The dataset has only 8 categories, so say why the result has fewer than 10 rows.
C4	Which products generate the highest revenue?	`SUM`, `GROUP BY`, `ORDER BY`	The top 10 products with category, units sold and revenue.
C5	What is the monthly revenue trend?	Date functions, `SUM`, `GROUP BY`, `ORDER BY`	24 rows in chronological order, each with a year-month label and revenue.
C6	Which cities generate the highest revenue?	`SUM`, `GROUP BY`, `ORDER BY`	One row per city (14 after cleaning), ranked by revenue.
C7	What is the average order value by city?	`AVG`, `GROUP BY`	One row per city with average order value. Note whether the highest-revenue city also has the highest average order value.
C8	Which payment method is used most frequently?	`COUNT`, `GROUP BY`, `ORDER BY`	One row per payment method with count and percentage of orders, ordered by count. State how you treated missing methods.
C9	What percentage of orders are cancelled or returned?	`CASE WHEN`, `COUNT`	Overall percentage for Cancelled, for Returned, and for the two combined.
C10	Which product categories have the highest return rates?	`CASE WHEN`, `GROUP BY`, `HAVING`, `ORDER BY`	One row per category with orders, returns and return rate, sorted by rate. Use `HAVING` to keep only categories with a stated minimum number of orders and justify the threshold.
C11	How does customer rating vary across product categories?	`AVG`, `MIN`, `MAX`, `COUNT`, `GROUP BY`	One row per category with average, minimum and maximum rating and the number of rated orders. Missing ratings must not count as zero.
C12	What is the average delivery time for each city?	`AVG`, `GROUP BY`	One row per city. Say which order statuses you included and why.
C13	Which cities have the highest average delivery time?	`AVG`, `HAVING`, subquery	The cities whose average delivery time exceeds the overall average, found using a subquery, ranked highest first.
C14	Analyse revenue by gender.	`SUM`, `COUNT`, `GROUP BY`	One row per gender group (including a group for missing gender) with orders, revenue, share of revenue and average order value.
C15	Analyse purchasing behaviour across age groups.	`CASE WHEN`, `COUNT`, `SUM`, `AVG`, `GROUP BY`	Rows for age bands you define (for example 18 to 24, 25 to 34, and so on, plus Unknown) with orders, revenue and average order value, in age order.
C16	Who are the top 10 customers by total spending?	`SUM`, `GROUP BY`, `ORDER BY`, `LIMIT`	10 rows with customer id, city, number of orders and total spend.
C17	Do higher-rated products or categories have higher sales?	`JOIN`, subqueries, `AVG`, `SUM`	One row per product with average rating, units sold and revenue, built by joining a rating summary to a sales summary. Add a 3 to 4 sentence conclusion. You may use Hive's `corr()` function to support it.

#	Question	Required HiveQL features	Expected output
C18	Which products have high sales volume but low revenue?	Subquery, `SUM`, `HAVING`	Products whose units sold are above the average units per product and whose revenue is below the average revenue per product. State your cut-offs.
C19	Which products have high revenue but relatively low sales volume?	Subquery, `SUM`, `HAVING`	Products above the average revenue per product and below the average units per product. State your cut-offs.
C20	Which month had the highest revenue?	Date functions, `ORDER BY`, `LIMIT` or subquery	One row with the year-month and its revenue. Also report the highest month within each year and suggest a business reason.

9. Part D: Visualization

Export selected Hive results to CSV files and build at least six charts from them. The charts must be built from Hive output, never from the raw dataset, so the analysis stays in Hive.

10.1 Exporting results from Hive

Both methods are acceptable. Keep every exported file and list it in your README.

```
# Method 1: beeline writes a CSV that includes a header row
beeline -u jdbc:hive2://localhost:10000/ecommerce_analytics \
        --outputformat=csv2 -f q_monthly_revenue.hql > monthly_revenue.csv
-- Method 2: write to a local folder from HiveQL (no header row is written)
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/hive_out/monthly_revenue'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT ... ;
```

If you use Method 2, add the column names yourself when you load the file, and say so in your README.

10.2 Tools

Use any one of: Python with Matplotlib or Seaborn (submit a notebook or script), Power BI (submit the `.pbix` file), or Tableau (submit the `.twbx` file). Use one tool for all charts.

10.3 Required visualizations

#	Chart	Chart type	Data comes from
V1	Monthly revenue trend	Line chart, one point per month in date order	Your C5 result
V2	Revenue by product category	Bar chart, sorted by revenue	Your C3 result
V3	Revenue by city	Bar chart (horizontal works well for 14 names) or a map	Your C6 result
V4	Payment-method distribution	Pie or donut chart	Your C8 result
V5	Order status distribution	Bar or pie chart	Your C2 result
V6	A chart of your choice that shows an insight you found in Part C or D	Any suitable type, and you justify the choice	Any Hive result you exported

Good choices for V6 include return rate by category, average delivery time by city, revenue by customer segment, month-over-month change, or rating against revenue by product. Pick the chart type that makes your insight easiest to see.

10.4 What every chart must have

1. A clear **title** that says what is shown and for which period.
2. **Axis labels** with units (for example *Revenue (INR)*), except on pie or donut charts.
3. A **legend** whenever there is more than one series or a colour carries meaning. A legend is not needed on a single-series chart.
4. A **short interpretation** of 2 to 3 sentences that describes what the chart shows.
5. **At least one business insight**: a conclusion the company could act on, such as where to invest, what to fix or what to investigate.

Put the interpretation and insight directly beneath the chart in your report.

10.5 Good practice

- Format large numbers so they are readable (for example INR in lakhs, or thousands separators).
- Sort bars by value unless the order has meaning, as in months or age bands.
- Do not use a pie chart for more than about six slices.
- Keep colours consistent across charts, and make sure the charts are readable in grayscale.

10. Final Analytical Report and Big Data Perspective

Submit one report as a PDF or Word file. .

11.1 Required report structure

#	Section	What it must contain
1	Introduction	The business problem, the goal of the analysis, and a one-paragraph summary of your approach.
2	Dataset description	Source, size, time period, fields, and the business definitions you used.
3	Hive architecture and workflow	A diagram or description of the path from CSV to HDFS or local storage, to Hive tables, to query results, to exported files, to charts. Name the Hive version and execution engine you used.
4	Hive database and table design	Database name, table definitions, data types and why you chose them, and the file format.
5	Data-loading procedure	The exact commands you ran and the validation checks that proved the load worked.
6	Data-cleaning procedure	The problems found, your treatment table (B7), and before-and-after evidence.
7	HiveQL queries	Every query for Parts B, C and D, with its number, copied from your `.hql` file.
8	Query outputs	The output of each query, as tables or screenshots, next to its query.
9	Visualizations	The six charts with titles, axis labels, legends, interpretation and business insight.
10	Interpretation of results	What the Part C and Part D results mean for the business, linked to the charts.
11	Key findings	The five to eight most important findings, each backed by a number.
12	Conclusion	A short summary and two or three concrete recommendations.

12. Deliverables and Submission

Submit one folder, zipped, named `<rollno>_hive_assignment.zip`, with the contents below.

12.1 Deliverables

#	Deliverable	File or folder	Notes
1	Dataset	`data/ecommerce_transactions.csv`	The original file, unchanged.
2	HiveQL script	`hql/<rollno>_assignment.hql`	All queries for Parts A to D in order, with question numbers as comments. It must run from top to bottom on a fresh Hive installation.
3	Query output files	`outputs/`	One file per question or per exported result, named by question number (for example `C5_monthly_revenue.csv`).
4	Visualization code or file	`viz/`	A Jupyter notebook or Python script, or a Power BI `.pbix` file, or a Tableau `.twbx` file.
5	Final analytical report	`report/<rollno>_report.pdf`	Structure in section 11.1.
6	README	`README.md`	Description in section 12.2.

12.2 README requirements

Your README must let a marker reproduce your analysis without contacting you. It must contain:

1. Your name and roll number.

2. The environment: operating system, Hive version, execution engine, how Hive was installed, and the Python or BI tool and its version.
3. The folder layout and one line about each file.
4. Step-by-step instructions, in order: how to start Hive, where to place the dataset, which command runs the `.hql` script, how the exports are produced, and how to run the notebook or open the BI file.
5. The paths or settings a marker must change (for example the `LOAD DATA` path).
6. The Part C feature checklist from section 8.2.
7. Assumptions and known limitations.

12.3 Pre-submission checklist

☐ The script runs end to end without edits other than the paths named in the README.

☐ Every number in the report can be traced to a query and an output file.

☐ All six charts are present and each has a title, axis labels, a legend where needed, an interpretation and an insight.

☐ The seven Big Data topics are answered, and the `EXPLAIN` output is included.

☐ Every query shown in the report matches the script.

☐ You have listed all sources and tools you used, including any generative AI assistance, in the README.